

Experiment 1

Title of practical -

Write problem statement for system/Project.

Objective of the Assignment :

Identify Project of enough complexity, which has at least 4-5 major functionalities.

Identify stakeholders, actors and write detail problem statement for your system.

Prerequisite :

Basic of software Engineering, Requirement Gathering, Relationships and Diagram

Theory :

Identify requirements :

Requirements in this context are the condition that a proposed solution or application must meet in order to solve the business problem. Identifying the requirements is not an exclusively technical process, and initially involves all the stakeholders, like representatives of the entity that has commissioned the software project, whom are not necessarily the technical team.

Together, they discuss and brainstorm about the problem, and decide what functions the proposed application or system must perform in order to solve it.

Functional Requirement -

specifies something that the application or system should do. often this is defined as a behaviour of the system that takes input and provides output. For example, a traveller fills out a form in an airline mobile application with his/her name and passport details, submits the form, and the application generates a boarding pass with the traveller's details.

Non-Functional Requirements, sometimes also called quality requirements, describe how the system performance, maintainability, and scalability, among many others. In the airline application example, the requirement that the application must display the boarding pass after a maximum five seconds from the time traveller presses the 'submit' button would be a non-functional requirement.

What is effective problem statement?

In project management, the problem statement is part of the project charter and defines what the problem is so that the project team and stakeholders can focus their attention on solving the problem.

Problem statement:-

The smart home automation system is an IOT driven solution that combines sensors, actuators, cloud computing and AI to automate and control home devices.

The system consist of :

- IOT enabled smart devices such as lights, fans, AC's, security cameras & smart locks.
- A central controller for managing device communication.
- cloud based services for remote monitoring and data storage.
- mobile and web applications for easy user access and control.

The system interact with :

- smart sensors : Detects motion, temperature humidity and gas leaks.
- Actuators & Relays : controls devices like lights, fans, AC's and door locks.
- cloud Platforms : stores data and executes automation commands.
- & voice assistants : works with google assistant, Alexa and siri for hands.

Product Functions -

The smart home automation provide following Functions:-

1] Device Control -

- Turn appliances ON/OFF via a smartphone.

2] security & surveillance

- Motion detection with real-time video feed from security cameras.

3] Energy management -

- monitor real time energy consumption.

4) Remote access & notification -

- control and monitor home automation appliance from anywhere.

Normal users -

The system is designed for -

1) Homewoners

- manage security, lighting & appliances remotely.

2) Elderly and disabled individuals -

- Use voice control for accessibility.

3) working professionals -

Automate devices for convenience & energy efficiency.

Operating environment :-

- Hardware level: microcontrollers, sensors, relays.

- software level: cloud-based services, mobile and web applications.

- supported devices:

Smartphones, tablets, and desktop browsers.

User characteristics:

~~Basic users:~~

~~Homeworkers who use mobile apps for control.~~

Advanced users:

Tech enthusiasts who customize automation

specific requirements:

1] Functional requirements -

① user authentication and access control.
- user must log into access controls.

② Device connectivity and control -
- connect and manage smart devices through mobile and web apps.
- Turn appliances ON/OFF remotely.

③ sensor data monitoring -
- display temperature, humidity and motion detection and.

④ security system features:
- send alerts for unauthorised access.
- Enabling remote video streaming for security cameras.

2] Non functional Requirements -

① Performance Requirements

- Devices response time should be under 2 second.

- Support 50+ connected devices per home.

2) security requirements :

- Implement end to end encryption.
- use Auth autentecation for secure logins.

3) Availability & Reliability -

- Ensure 24/7 operations with 99.9% uptime.

Software Requirements -

1] FrontEnd Development : HTML, CSS, JS

2] Backend development - Node.js, python, firebase.

3] Database : MySQL, MongoDB, fireBase, Realtime, database.

4] ~~cloud~~ integration : AWS IoT, Google cloud IoT, Thing speak.

5] Mobile App development : Flutter, React Native, kotlin.

6] ~~Mobile~~ Communication Protocols :

MQTT, HTTP, websockets.

Hardware Requirements :

- 1) micro-controllers, ESP 8266, Raspberry Pi, Arduino.
- 2) sensors: Motion, temperature, humidity, gas, smoke, detection.
- 3) Actuators: Relays for controlling appliances.

Conclusion :

The smart Home Automation system enhances convenience, security and energy efficient through IOT, cloud computing and AI, with features like remote control, voice commands and automation rule.

Practical No: 2

Title : Prepare a use case model, from the given description using UML 2 notations.

Objectives : Identify major use cases, Identify actors. Write use case specification for all major use cases. Draw detail use case Diagram using UML 2.0 notations.

prerequisite : Basic UML notations and UML Things, Relationships and Diagrams.

Theory :

1. Use case Diagram :-

Use case diagrams are usually referred to as behavior diagrams used to describe a set of actions (use cases).

1.1 purpose of use case diagram :

- used to gather requirements of a system. used to get an outside view of a system.
- Identify external and internal factors influencing the system.
- show the interacting among the requirements are actors.

1.2 contents of use case diagram :

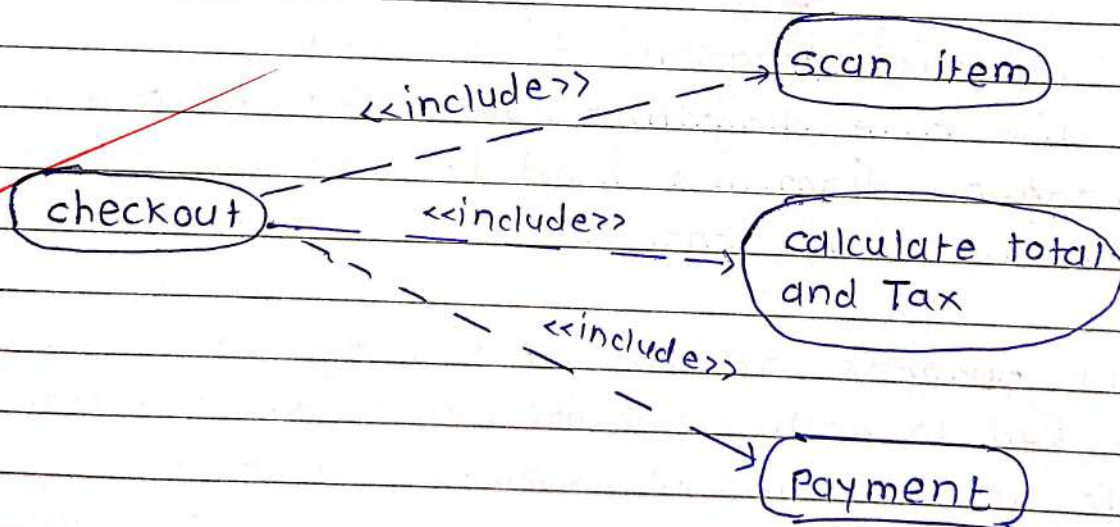
- use case
- Actors

• Relationships among the use cases and actors.

1.2.1. Use cases : use cases are nothing but the system functionalities written in an organized manner.

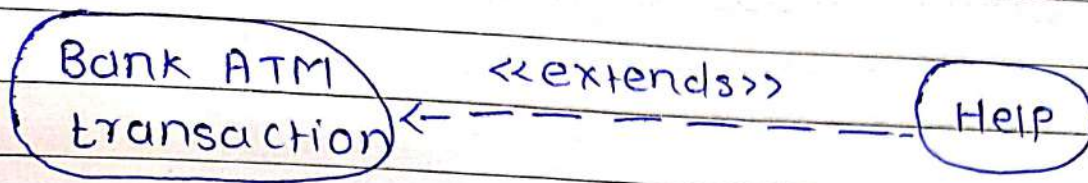
1.2.2. Actors : Actors can be defined as something that interacts with the system.

1.2.3. Include Relationship : An include relationship between use cases means that the base case explicitly incorporates the behavior of another use case of a location specified in the base.



• Extend Relationship :-

An extend relationship between use cases means that the base use case implicitly incorporates the behaviour of another use case at location specified indirectly by extending the use case.

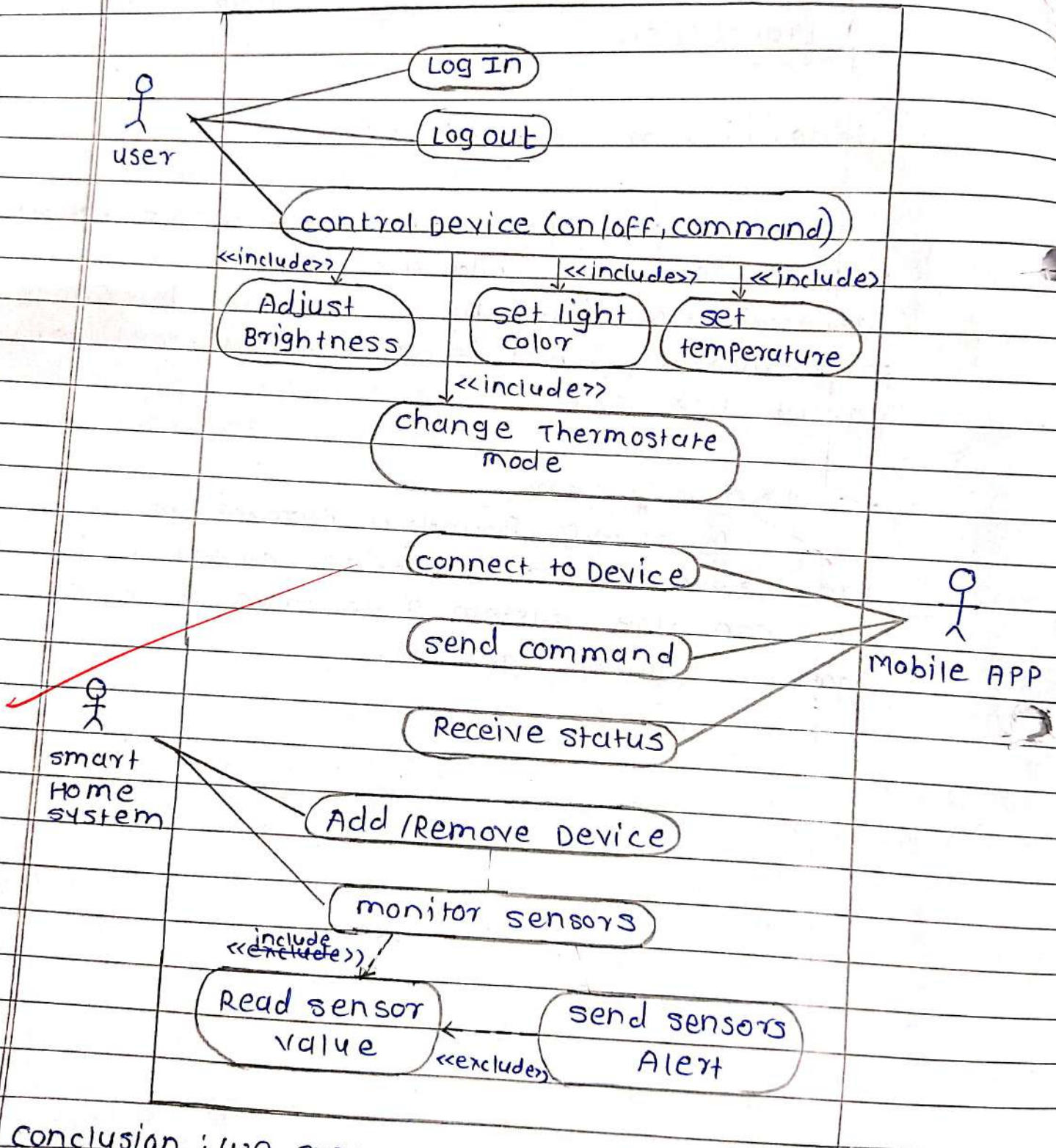


Generalization Relationship :-

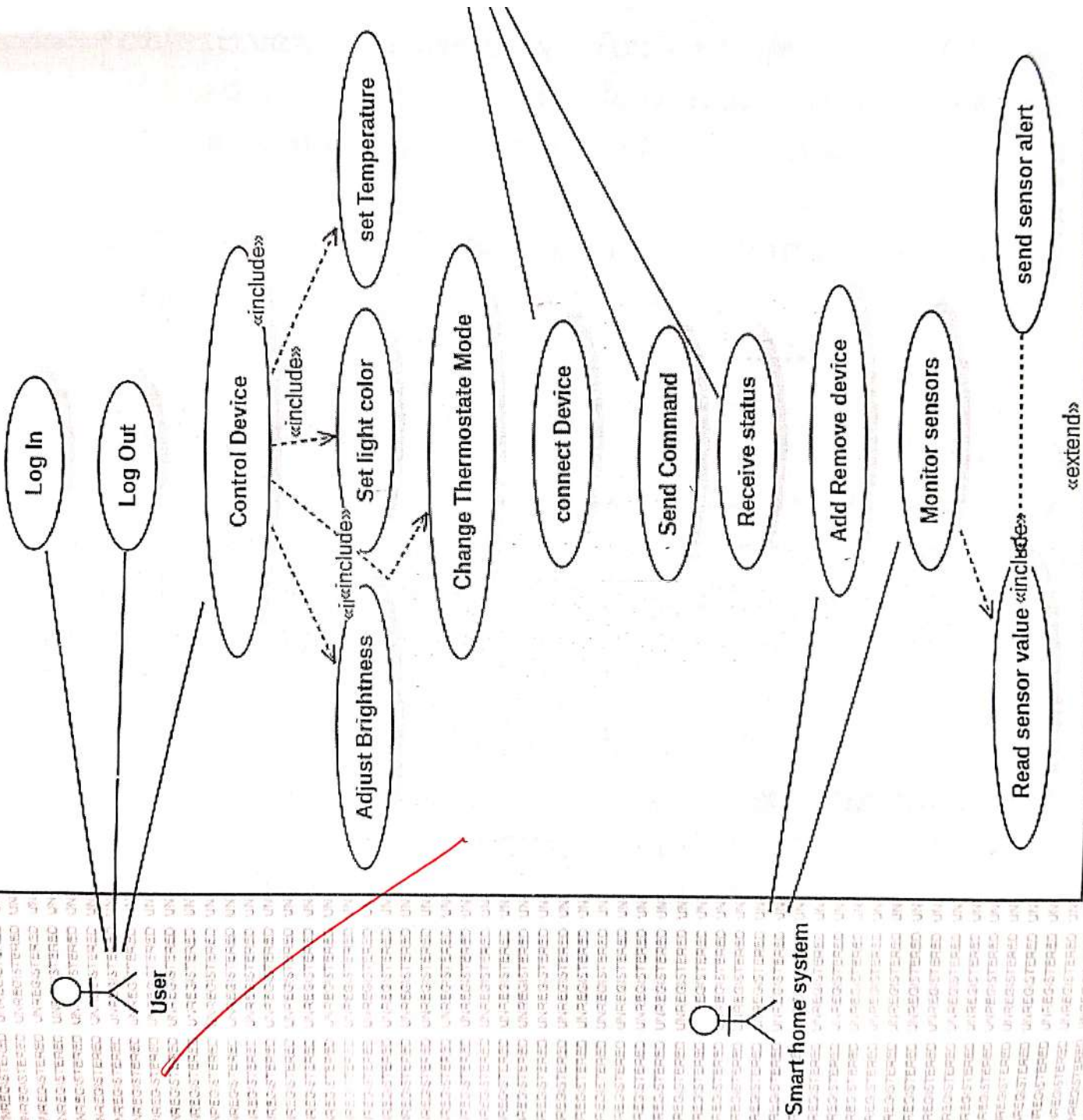
It is also a parent-child relationship between use cases. The child use case in the generalization has the underlying business process meaning, but is an enhancement of parent use case.

• ~~System Boundary~~:

A system Boundary element is a non-UML element used to define conceptual boundaries. You can use system Boundaries to help group logically related.



✓ Conclusion : we prepare a use case model, which describes the various users and functionalities.



Handwritten signature

Experiment No: 3

Title : Prepare Activity model from the given description using UML 2 notations.

Objectives : Identify Activity states and Action states. Draw Activity diagram with swim lanes using UML 2.0 Notations for major use cases.

Prerequisite : Basic UML Notations and UML Things, Relationships and Diagrams.

Theory :

1. Activity Diagram :

Activity diagrams are used to model the dynamic aspects of a system. An activity diagram is essentially a flow chart showing flow of control from activity to activity.

2. Purpose of Activity diagrams :

Activity diagrams are used to typically two ways

1. To model a work flow : Work flow often lie on fringe of software intensive system and we use to visualize, specify, construct and document business processor that involve the system of that is being developed.

2. To model an operation :

We use flow chart like activity diagrams to model details of a computation. Here modeling

of branch, fork and join states are particularly important.

3. contents of an activity diagram;

Activity diagram commonly contains.

1. Activity, states action states.
2. Transitions
3. Objects

3.1 Action states: An action states represents the non-interruptible action of objects. You can draw an action state using a rectangle with rounded corners.

3.2 Activity state

Activity state can be further decomposed; their activity being represented by other activity diagram.

3.3 Transitions:

1. When the action or activity of state completes, flow of control passes immediately to next action or activity state.
2. This flow is specified by using transition to show path from one activity to state to the action or activity state.

3.4 Branching :

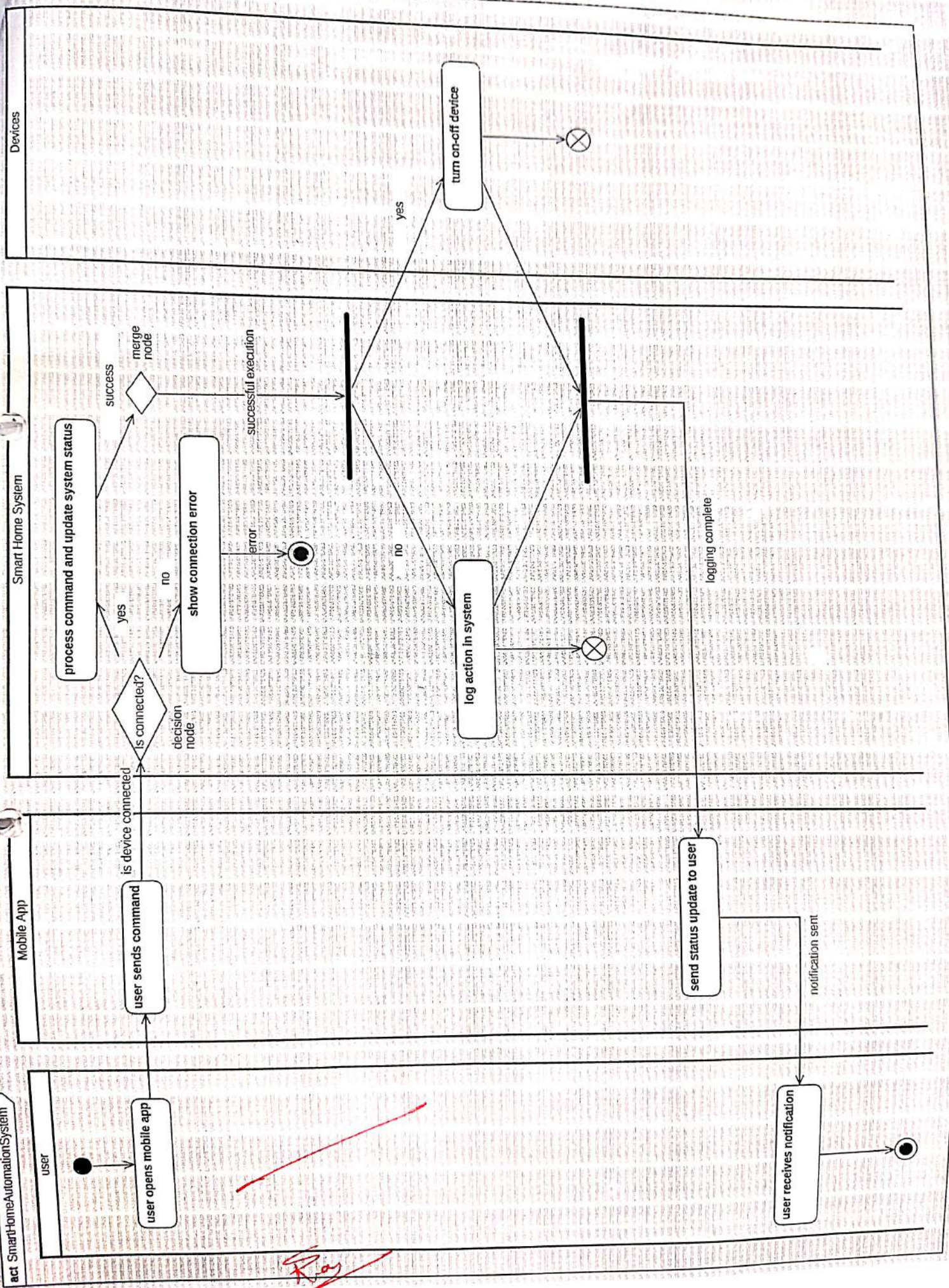
1. Simple sequential transitions are common. But they are not kind of path we need to module.
2. As in Flow chart a branch maybe included to show any alternate flow if required.
3. These alt paths are taken on evaluation on boolean expression.
4. A branch may have the one incoming transition and two or more outgoing ones.
5. The effect of iteration can be achieved by using one action state that sets value of an iterator

3.5 Forking and Joining.

1. Simple branching, sequential transitions are most path found in activity diagrams but while modeling work flows of bussiness processors that are concurrent, may be.
2. Synchronization bars are used to specify the forking the joining of parallel flow of control.

Conclusion :

we prepare activity model to show activities, to model the dynamic aspects of a system and for showing flow of central activity.



Experiment No: 4

Title of the practical :

prepare a class diagram from the given problem description using UML 2.0 and implement the class diagram with a suitable object oriented language.

Objective of the Assignment -

prepare Identify Analysis classes & assign responsibilities. Prepare Data Dictionary. Draw Analysis class model using UML 2.0 Notations. Implement Analysis class Model - class diagram with a suitable oo concept.

Theory -

1. Model :

1.1 Introduction : model is an abstract of something for the purpose of understanding, before building it. Software designers build many kinds of models for various purpose before constructing actual software system.

Examples -

1. Architecture models to show customers.
2. Pencil sketches for composition of oil painting.
3. Blueprints of machine parts.
4. Storyboard of advertisements.
5. Outline of books.

1.2 Analysis model :

The purpose of any Analysis Activity in sw life cycle is to create a model of the system's functional requirements that is independent of implementation constraints.

This model:

- Defines what the system does not how it does it.
- Defines logical requirements in more detail than the use case model.
- Leaves out all technology detail, including system topology.

For example, data may be modeled by ER diagrams, and behaviours by flow charts.

The primary tasks in analysis model (OOA) are

1. class diagram;

• gives an overview of a system by showing its classes and the relationship among them.

• The diagram shows attributes and operation of each class to way to describe the overall architecture of system components.

• Generally uml diagrams are not directly mapped with any oop languages but the class diagram is an exception.

Each class is represented by a rectangle
subdivide into three components.

- Name
- Attributes
- operations.

Modifiers are used to include visibility of attributes and operations.

'+' is used to public visibility.

'#' for protected visibility

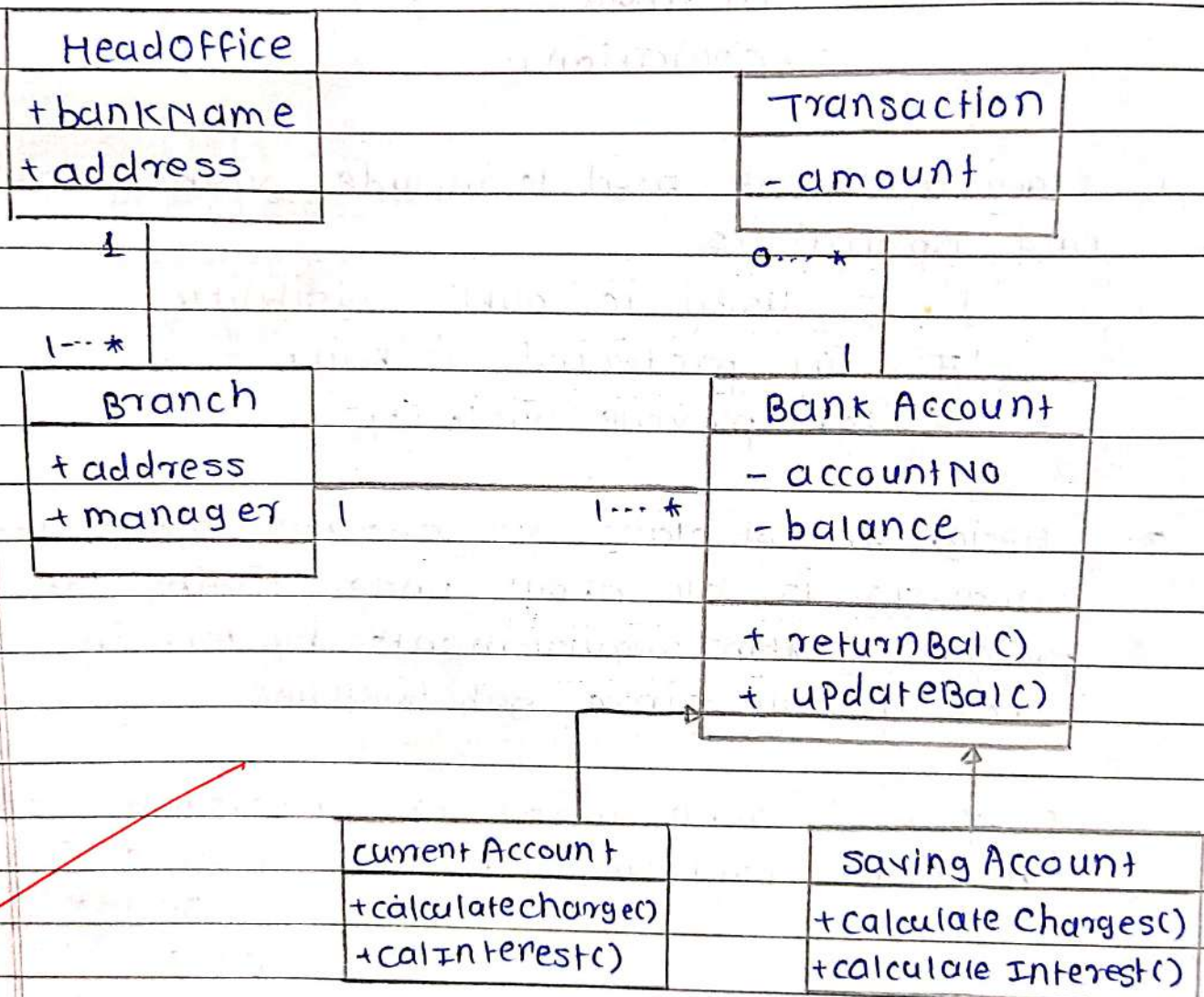
'-' for private visibility.

3. Design model class vs analysis model class.
analysis is all about understanding of requirements problems within requirements by breaking the whole module into sub modules.

~~Diagrams~~ can be made to understand and pin point problems in requirements - design

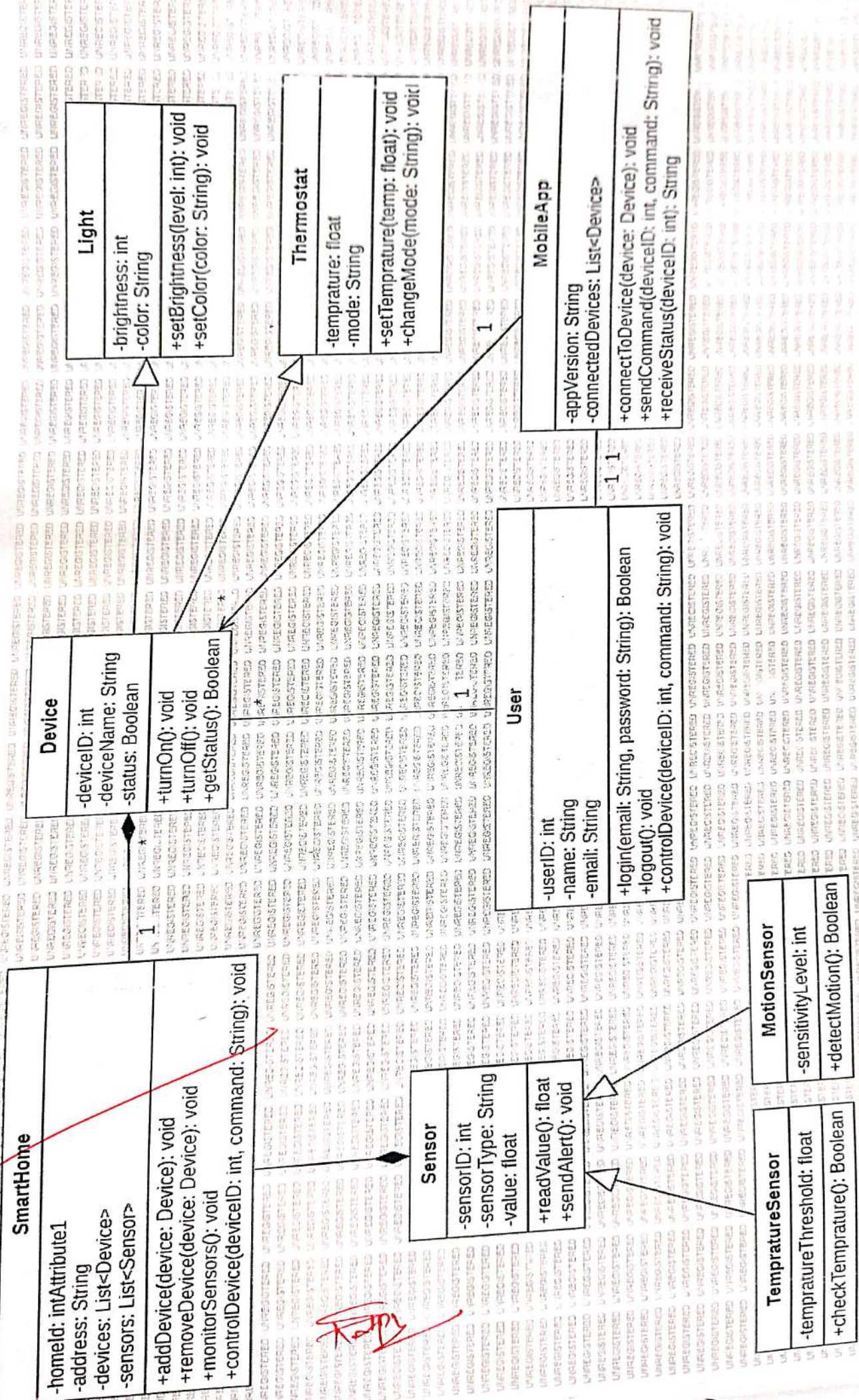
Analysis		order
	order	- deliveryDate : Date
	Placement order	- orderNumber : int
	Delivery Date	- PlacementDate : Date
	order Number	- taxes : currency
	calculate Total	- total : currency
	calculate Taxes	# calculate Taxes : currency
		# calculateTotal() : currency

consider the example -: A banking system has a variety of classes, which can be quickly outlined with class Diagram.



Conclusion :

In this way we have studied basics of design model and implemented design model using object oriented language Java.



Experiment No ; 5

Title of the Practicle :

prepare a sequence Diagram From the given problem description using UML 2.0 & implement the sequence diagram with a suitable object oriented language.

Objective of the Assignment : Identify at least 5 major scenarios for your system. draw sequence Diagram for every scenario by using advanced notations using UML 2.0 Implement these scenarios by taking reference of design model implementation using suitable object-oriented language.

Pre requisite : Basic object oriented concepts.

Theory :

1. Sequence Diagram

sequence Diagram is the most common kind of interaction diagram, which focuses on the message interchange betⁿ a number of lifelines.

sequence diagram describes an interaction by focusing on the sequence of messages that are exchanged, along with their corresponding occurrence specifications on the lifelines.

1.1 Contents of sequence Diagram :

1. Objects

2. Messages

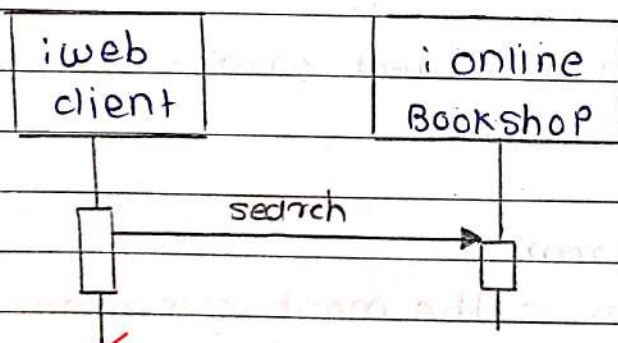
3. Interaction Fragments.

1.1.2 Message:

It is a named element that defines one specific kind of communication between lifelines of interaction. The message specifies not only the kind of communication but also the sender & the receiver.

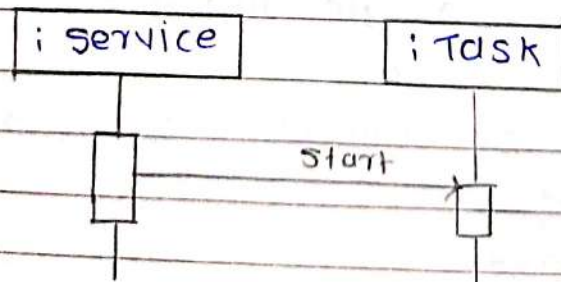
a. Synchronous call typically represents operation call - send message and suspend execution while

b. waiting for response. Synchronous call messages are shown with filled arrow head.

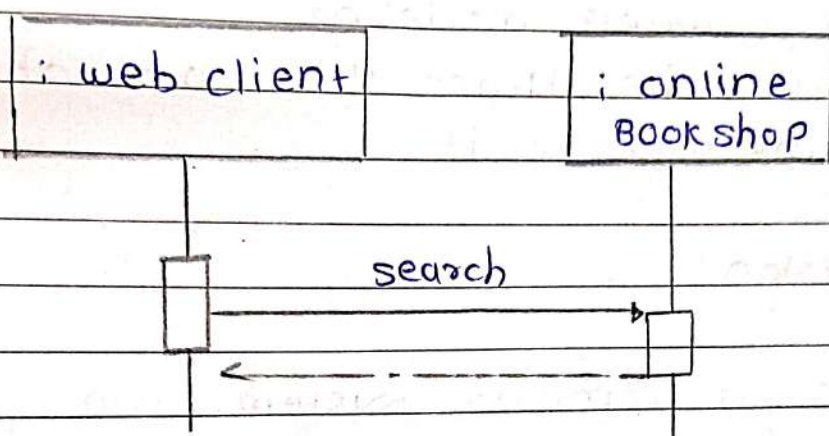


c. Asynchronous call - send messages and proceed immediately without waiting for return value.

Asynchronous msg have an open arrow head.



d. Return message: It is a reply message to an operation call is shown as a dashed line with open arrow head.



e. create message is sent to a lifeline to create itself. It is shown as a dashed line with open arrowhead (looks the same as reply message).

f. delete message is sent to terminate another lifeline. The lifeline usually ends with a cross in the form of ~~x~~ at the bottom denoting destruction occurrence.

1.2 Interaction Operation :

A combined flow is used to group set of messages together to show conditional flow in sequence diagram.

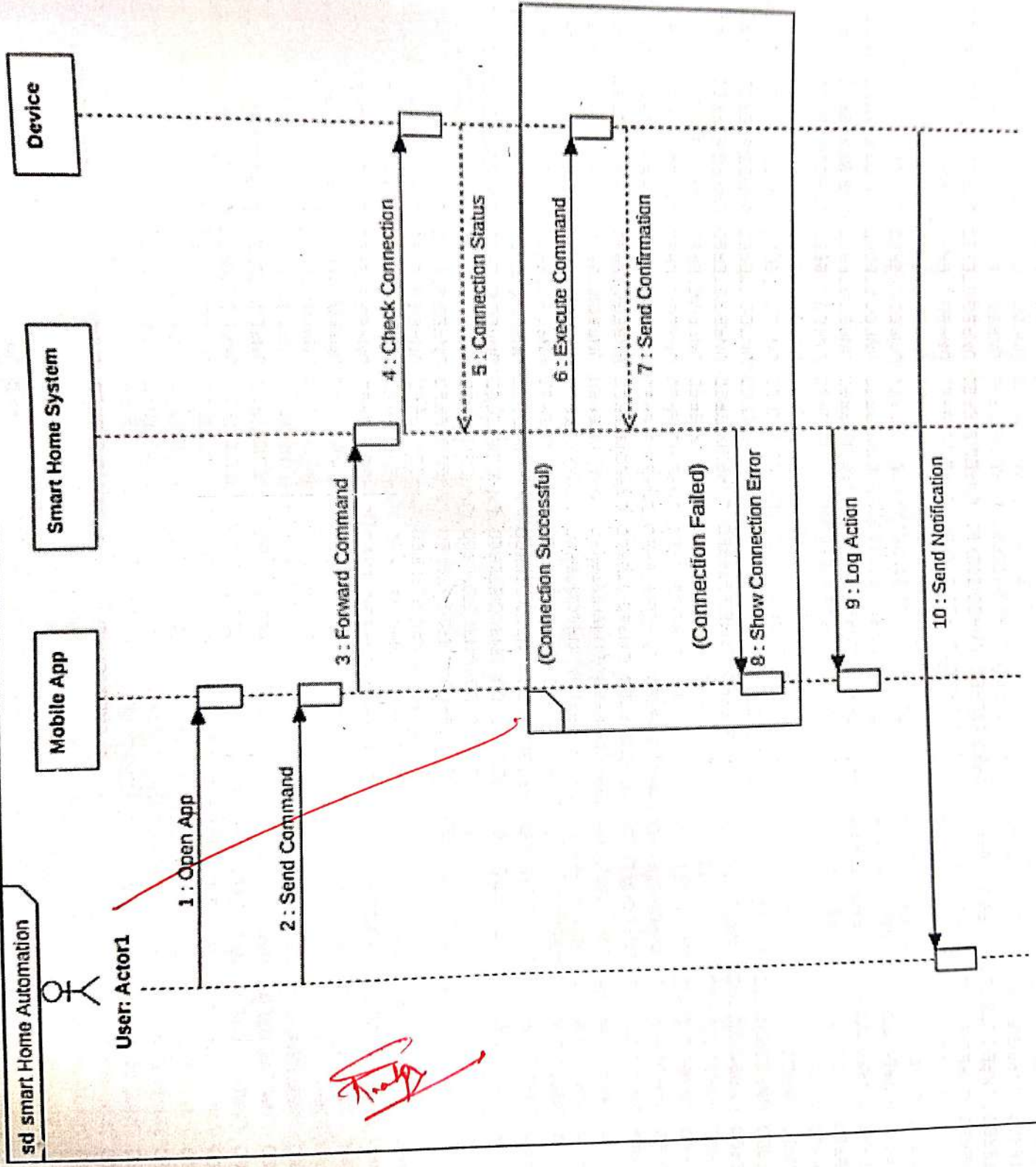
a. Alteration : fragment is used to model a sequence diagram in case if alternative solution is present.

d. parallel : IF two activities are going simultaneously, then parallel ~~pr~~ fragment is used.

d. loop : when modeling a sequence diagram, there will be times that an object will need to send message to it.

Conclusion :

For a complex system, we prepared a use case model, sequence model and activity model.



Handwritten signature

Experiment No; 6

Title : Prepare a state model from the given problem description, draw a state diagram using UML notations and implement the state model with a suitable object oriented language.

Objective :

Identify the state and event for your system.

1. Study state transitions and identify Guard conditions.
2. Draw state chart diagram with advanced UML notations.
3. Implement the state model with a suitable object-oriented language.

Prerequisite : Basic object oriented concepts.

Theory :

According to view of a system there are three models namely,

1. class model
2. State model
3. Interaction model.

1. state model -

State model describe those aspects for objects concerned with time and sequencing of operation. This model captures control of systems.

2. State diagram.

State diagrams are used to describe the behaviour of a system. These diagrams describe all of the possible states of an object through the system.

2.1 State machine:

A state machine is a behavior that specifies the sequences of state an object goes through during its finite time in response to events.

a. state

a state is a condition of an object in which it performs some activity or waits for an event.

b. denoted by rectangle with rounded corners and components.

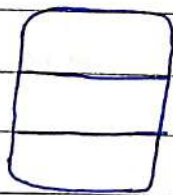


Fig 1 Graphical Notation for state

A state has following things associated with it.

Name A textual string which distinguishes the state from other states; a state may also be anonymous, meaning that it has no name.

Entry / exit actions	Actions executed on entering and exiting the state.
Internal transitions	Transitions that are handled and exiting the state.
Substates	The nested structure of state involving disjoint or concurrent sub states.

c. Exit Actions

Entry and exit actions allow the same action to be dispatched every time the state is entered or left, respectively.

d. Internal Transitions.

Internal transitions allow events to be handled within the state ~~without~~ leaving the state.

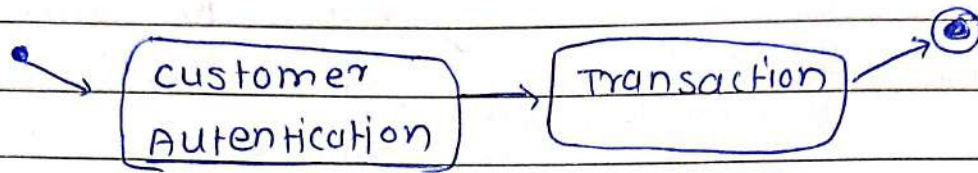
f. Deferred Events:

Deferred events are those whose handling is postponed until a state in which the event is not deferred becomes active.

g. Composition state - A state that consists of sub states is called as composition state

h. Region: A region is an orthogonal part of either a composite state or a state machine. It contains states and transitions.

Serving customer



j. Concurrent sub state :

A concurrent substate is a substate in which in which an object can reside simultaneously with other sub states at that same level within the given composite state.

k. ~~Final~~ state.

The end of the state diagram is shown by a bull's eye symbol, also called a final state.

Conclusion ;

So we can conclude that a state diagram sometimes know as the state machine diagram, is a type of behaviour diagram in the unified modeling language (UML) that shows transitions between various objects.

Idle

On user command

Monitoring

fault

Error State

on device sensor fault

Auto control

ON

Device ON

Device ON

ON

Device OFF

on sensor trigger or schedule

on issue resolved

Flow

Flow

Experiment No: 7

Aim: Prepare deployment diagram using UML notation.

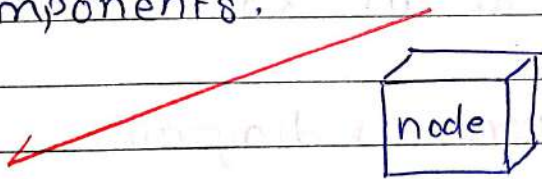
Theory: what is deployment diagram?

A deployment diagram is a UML diagram type that shows the execution architecture of the system, including nodes such as hardware or software execution environment, and the middleware connecting them.

Deployment diagrams are typically used to visualize the physical hardware and software of a system using which if you can understand how the system will be physically deployed on the hardware.

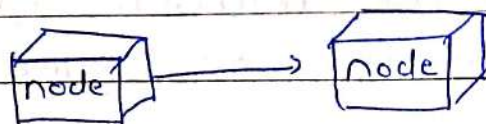
Deployment diagram notation:

Node - A node represented as a cube is a physical entity that executes one or more components.



Artifacts - These are concentrate elements that are caused by development process.

Communication association -



This is represented by a solid line between two nodes. It shows the path of communication between nodes.

devices ;

A device is a node that is used to represent a physical computational resource in a system.

- + Attribute 1 : Type
- + Attribute 2 : Type
- + Attribute 3 : Type
- + operation 1 : Return
- + operation 1 : Return
- operation 1 : Return

~~deployment specification -~~

deployment specification is a configuration file such as text file or an XML document.

How to draw deployment diagram -

1) identify a purpose of your deployment diagram and to do so you need to identify nodes and devices within the system you will be visualizing the diagram.

2) figure out the relationship between the nodes and device once you know they are connected.

ported to hold the communication association to the diagram.

3] Identify what other elements are like components, active, objects you need to add to complete the diagram.

4] Add dependencies between component and objects are required.

Conclusion :

For a complex system we purpose deployment diagram to show the structure s/w , h/w connection.

~~Final~~

